

AcquirerX Deep-Dive 3/10 — Auth Service

Service: `auth-service` **Port:** `9099` **Database:** `acquirerx_auth` **Role:** Identity & access management — users, JWT issuance, audit logging, profile (IAM)

1. What It Is

Auth-service owns everything related to **who you are and what you can do**:

- User registration and login
- Password hashing (BCrypt)
- JWT token generation
- User CRUD (admin can deactivate, change role)
- Profile management (`/iam/me`)
- Login history
- Audit logging — who did what, when

Every other service depends on auth-service indirectly: the gateway uses the same JWT secret to validate tokens issued here, and downstream services trust the `X-Username` / `X-User-Role` headers the gateway derives from those tokens.

2. Why It's a Separate Service

In a monolith, auth would be a package. In microservices, it's its own service because:

- **Single source of truth** — one database for users, no syncing across services
 - **Independent scaling** — login is high-traffic; can scale auth-service alone
 - **Security isolation** — only auth-service touches password hashes; even if another service is compromised, hashes don't leak
 - **Independent deployment** — fixing a JWT bug shouldn't require redeploying merchant-service
-

3. Tech Stack

- **Spring Boot 3.x**
 - **Spring Security 6** — for filter chain, BCrypt encoder, authentication manager
 - **Spring Data JPA + MySQL** — for persistence
 - **JWT 0.11.x** — for JWT generation
 - **Lombok** — for `@RequiredArgsConstructor`, `@Slf4j`, `@Getter`/`@Setter`
-

4. Code Structure

```
auth-service/src/main/java/com/acquirerx/auth/
├─ AuthServiceApplication.java
├─ config/
│   ├─ CorsConfig.java
│   ├─ OpenApiConfig.java
│   ├─ SecurityConfig.java
│   └─ security/
│       ├─ JwtFilter.java
│       └─ JwtUtil.java
├─ controller/
│   └─ AuthController.java
├─ dto/
│   ├─ LoginRequestDTO.java / LoginResponseDTO.java
│   ├─ RegisterRequestDTO.java
│   ├─ UserResponseDTO.java / UserFilterDTO.java
│   └─ AuditLogResponseDTO.java / AuditFilterDTO.java
├─ entity/
│   ├─ User.java
│   └─ AuditLog.java
├─ repository/
│   ├─ UserRepository.java
│   └─ AuditLogRepository.java
└─ service/
    ├─ AuthService.java
    └─ AuditService.java
```

5. Entities

5.1 `User` (table: `app_user`)

Why `app_user` and not `user`? Because `user` is a reserved keyword in many SQL dialects (PostgreSQL, MySQL in some modes), so prefixing avoids surprise errors.

Fields:

Field	Type	Purpose
<code>id</code>	Long	PK, auto-increment
<code>username</code>	String, unique	Login identifier
<code>passwordHash</code>	String	BCrypt hash, never plaintext
<code>email</code>	String	Contact address
<code>role</code>	String	One of ADMIN/RISK/DISPUTES/RECON/MERCHANT_OPS
<code>status</code>	String	ACTIVE / INACTIVE / PENDING
<code>createdAt</code>	LocalDateTime	Auto-set on creation
<code>updatedAt</code>	LocalDateTime	Auto-updated
<code>lastLoginAt</code>	LocalDateTime	Updated on every successful login

5.2 `AuditLog` (table: `audit_log`)

Append-only log of sensitive actions across the system.

Field	Type	Purpose
<code>auditId</code>	Long	PK
<code>actorUsername</code>	String	Who did the action
<code>action</code>	String	What — e.g., <code>USER_CREATED</code> , <code>ROLE_CHANGED</code> , <code>LOGIN_FAILED</code>
<code>targetType</code>	String	What kind of resource — e.g., <code>USER</code> , <code>MERCHANT</code>
<code>targetId</code>	String	ID of the affected resource (string to support any ID type)
<code>details</code>	String/Text	Human-readable details
<code>performedAt</code>	LocalDateTime	When

Why string for `targetId`: different services use Long IDs, UUIDs, etc. Storing as string keeps the audit log generic.

6. Security Configuration

6.1 `SecurityConfig.java` — what it does

Configures Spring Security's filter chain. Typical `AcquirerX` shape:

```
java
```

```

@Configuration
@EnableWebSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtFilter jwtFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())
            .cors(cors -> {}) // use CorsConfig bean
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STRICT))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/auth/login", "/api/v1/auth/register",
                    "/api/v1/auth/validate", "/v3/api-docs/**",
                    "/swagger-ui/**", "/actuator/**").permitAll()
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
            .build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config)
        throws Exception {
        return config.getAuthenticationManager();
    }
}

```

Why each setting:

Setting	Reason
<code>csrf().disable()</code>	CSRF protection is for cookie-based auth. JWT in headers isn't vulnerable to CSRF
<code>STATELESS</code> session	No <code>HttpSession</code> is created — JWT carries identity per-request
<code>permitAll()</code> for login/register	These need to be reachable without a token (chicken-and-egg)
<code>anyRequest().authenticated()</code>	Everything else needs a valid JWT
<code>JwtFilter</code> before <code>UsernamePasswordAuthenticationFilter</code>	<code>JwtFilter</code> runs first to populate <code>SecurityContext</code>
<code>BCryptPasswordEncoder</code>	Adaptive hashing with salt — slow by design to thwart brute force

6.2 `JwtFilter.java` — request filter

This filter runs once per request (extends `OncePerRequestFilter`) and:

1. Reads `Authorization` header
2. If absent or doesn't start with `Bearer` → continues without authenticating (Security's `anyRequest().authenticated()` will reject it later)
3. Extracts the token, calls `JwtUtil.isValid()` → if invalid, continues without authenticating
4. Extracts username and role from token
5. Builds an `Authentication` object and puts it in `SecurityContextHolder`
6. Calls `chain.doFilter()` to continue

After this filter runs, downstream Spring Security can read

`SecurityContextHolder.getContext().getAuthentication()` to know who the user is.

```
java
```

```

@Component
@RequiredArgsConstructor
public class JwtFilter extends OncePerRequestFilter {

    private final JwtUtil jwtUtil;

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
                                    FilterChain chain) throws ServletException, IOException {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            if (jwtUtil.isValid(token)) {
                String username = jwtUtil.extractUsername(token);
                String role = jwtUtil.extractRole(token);
                var auth = new UsernamePasswordAuthenticationToken(
                    username, null,
                    List.of(new SimpleGrantedAuthority("ROLE_" + role))
                );
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }
        chain.doFilter(req, res);
    }
}

```

Note: Now that JWT validation is centralized at the gateway (Phase E), this filter is technically redundant for production — gateway pre-validates. But it remains as defense-in-depth for cases where someone hits auth-service directly during local testing.

6.3 `JwtUtil.java` — full implementation (this is the one you provided)

java

@Component

```
public class JwtUtil {

    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration}")
    private long expiration;

    private SecretKey getKey() {
        return Keys.hmacShaKeyFor(secret.getBytes());
    }

    public String generateToken(String username, String role) {
        return Jwts.builder()
            .setSubject(username)
            .claim("role", role)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + expiration))
            .signWith(getKey(), SignatureAlgorithm.HS256)
            .compact();
    }

    public String extractUsername(String token) { return getClaims(token).getSubject(); }
    public String extractRole(String token) { return getClaims(token).get("role", String.class); }

    public boolean isValid(String token) {
        try { getClaims(token); return true; }
        catch (JwtException | IllegalArgumentException e) { return false; }
    }

    private Claims getClaims(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(getKey())
            .build()
            .parseClaimsJws(token)
            .getBody();
    }
}
```

Line-by-line:

- `@Value("${jwt.secret}")` — reads from `application.properties`. Same secret as gateway.
- `Keys.hmacShaKeyFor(secret.getBytes())` — converts string secret into a `SecretKey` object suitable for HS256
- `setSubject(username)` — JWT standard claim `sub` = username
- `.claim("role", role)` — custom claim, this is how we pass the role around
- `setIssuedAt(new Date())` — `iat` claim
- `setExpiration(...)` — `exp` claim, 24 hours by default
- `signWith(key, HS256)` — HMAC-SHA-256 signing
- `compact()` — serializes the signed JWT into the compact `header.payload.signature` string format
- `parserBuilder().setSigningKey(key).build()` — parser configured to verify against the same secret
- `parseClaimsJws(token)` — parses AND validates signature in one call. Throws if invalid.

JWT structure produced:

```
Header (base64url): {"alg":"HS256","typ":"JWT"}
Payload (base64url):
{"sub":"admin","role":"ADMIN","iat":1730000000,"exp":1730086400}
Signature: HMAC-SHA-256(header + "." + payload, secret)
```

6.4 `CorsConfig.java`

Auth-service has its own CORS config because direct calls (e.g., during local testing without going through gateway) need CORS too. Mirrors the gateway's config — same allowed origins, methods, etc.

6.5 `OpenApiConfig.java`

Configures Swagger UI to support JWT — the "Authorize" button at the top of `/swagger-ui.html`. Adds a `Bearer` security scheme so you can paste a token in Swagger and have it auto-attached to requests.

7. The Auth Service

7.1 AuthService.java — typical structure

Handles register, login, and user CRUD. Skeleton:

```
java
```

```
@Service
@RequiredArgsConstructor
public class AuthService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtUtil jwtUtil;
    private final AuditService auditService;
    private final AuthenticationManager authManager;

    public void register(RegisterRequestDTO dto) {
        if (userRepository.existsByUsername(dto.getUsername()))
            throw new IllegalStateException("Username already taken");

        User user = new User();
        user.setUsername(dto.getUsername());
        user.setPasswordHash(passwordEncoder.encode(dto.getPassword()));
        user.setEmail(dto.getEmail());
        user.setRole(dto.getRole());
        user.setStatus("ACTIVE");

        User saved = userRepository.save(user);
        auditService.logAction("system", "USER_REGISTERED", "USER",
                               saved.getId().toString(), "username=" + dto.getUsername());
    }

    public LoginResponseDTO login(LoginRequestDTO dto) {
        User user = userRepository.findByUsername(dto.getUsername())
            .orElseThrow(() -> new BadCredentialsException("Invalid credentials"));

        if (!"ACTIVE".equals(user.getStatus()))
            throw new IllegalStateException("Account is " + user.getStatus());

        if (!passwordEncoder.matches(dto.getPassword(), user.getPasswordHash())) {
            auditService.logAction(dto.getUsername(), "LOGIN_FAILED", "USER",
                                   user.getId().toString(), "Bad password");
            throw new BadCredentialsException("Invalid credentials");
        }

        String token = jwtUtil.generateToken(user.getUsername(), user.getRole());
        user.setLastLoginAt(LocalDate.now());
        userRepository.save(user);
        auditService.logAction(user.getUsername(), "LOGIN_SUCCESS", "USER",
```

```

        user.getId().toString(), null);

        return new LoginResponseDTO(token, user.getUsername(), user.getRole(), "Logi
    }
}

```

Key design decisions:

- **BCrypt for passwords** — never store plaintext. BCrypt is adaptive (you can increase work factor over time)
- `existsByUsername` **check before insert** — gives a clear error message rather than relying on DB unique constraint exception
- **Status check on login** — INACTIVE users can't log in even with right password
- **Audit on every login attempt** — both success and failure, useful for security monitoring
- **Update** `lastLoginAt` — used in profile UI to show "last seen at" info

7.2 User CRUD methods (admin-only)

- `getAllUsers(pagination)` — paginated list
- `getUserById(id)`
- `searchUsers(filter, pagination)` — by username, role, status, email
- `updateRole(id, newRole)` — admin can change a user's role
- `deactivate(id)` — sets status=INACTIVE, user can no longer log in
- `reactivate(id)` — sets status=ACTIVE

Each of these calls `auditService.logAction(...)` so admin actions are traceable.

8. Audit Service

8.1 `AuditService.java` — full implementation (you provided this)

```

java

```

```

@Service
@RequiredArgsConstructor
public class AuditService {

    private final AuditLogRepository auditLogRepository;

    public void logAction(String actorUsername, String action,
                        String targetType, String targetId, String details) {
        AuditLog entry = new AuditLog();
        entry.setActorUsername(actorUsername);
        entry.setAction(action);
        entry.setTargetType(targetType);
        entry.setTargetId(targetId);
        entry.setDetails(details);
        auditLogRepository.save(entry);
    }

    public PagedResponseDTO<AuditLogResponseDTO> getAllLogs(PaginationParams pagination) { ... }
    public PagedResponseDTO<AuditLogResponseDTO> searchLogs(...) { ... }
    public PagedResponseDTO<AuditLogResponseDTO> getLogsByActor(...) { ... }
}

```

Why audit logging matters for AcquirerX:

Payment systems are heavily regulated. Auditors will ask: "who deactivated merchant X on October 5?" or "show me everyone who accessed cardholder Y's transactions." Without an audit log, those questions are unanswerable.

Why a separate `AuditService` (not just inline DB writes):

- Single concern — all audit logic in one place
- Easy to swap out (e.g., later switch from MySQL to a dedicated audit log system like an immutable event store)
- Other services could call this via Feign if cross-service auditing becomes a need

8.2 Sortable fields (defensive validation)

```
java
```

```
private static final Set<String> SORTABLE_FIELDS = Set.of(
    "auditId", "actorUsername", "action", "targetId", "performedAt"
);
```

Before applying a sort, the pagination layer validates that the requested `sortBy` field is in this whitelist. This prevents:

- SQL injection via sort parameter
 - Sorting by random non-indexed columns (perf disaster)
 - 500 errors from typos in field names
-

9. Repositories

9.1 `UserRepository.java`

```
java

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);
    boolean existsByUsername(String username);
    Page<User> findByFilters(...filters, Pageable pageable);
}
```

`JpaRepository` gives `findAll`, `save`, `delete`, etc., for free. Custom finders use Spring Data's method naming convention.

9.2 `AuditLogRepository.java`

```
java
```

```

public interface AuditLogRepository extends JpaRepository<AuditLog, Long> {
    Page<AuditLog> findByActorUsername(String actor, Pageable pageable);

    @Query("SELECT a FROM AuditLog a WHERE " +
        "(:actor IS NULL OR a.actorUsername = :actor) AND " +
        "(:action IS NULL OR a.action = :action) AND " +
        "(:targetId IS NULL OR a.targetId = :targetId) AND " +
        "(:from IS NULL OR a.performedAt >= :from) AND " +
        "(:to IS NULL OR a.performedAt <= :to)")
    Page<AuditLog> findByFiltersPaged(@Param("actor") String actor,
        @Param("action") String action,
        @Param("targetId") String targetId,
        @Param("from") LocalDateTime from,
        @Param("to") LocalDateTime to,
        Pageable pageable);
}

```

The `(:param IS NULL OR field = :param)` pattern is the standard way to do "optional filters" in JPQL — if the user doesn't provide a filter, the condition becomes a no-op.

10. Controllers

10.1 `AuthController.java`

Maps HTTP routes to service methods. Skeleton:

```

java

```

```

@RestController
@RequestMapping("/api/v1/auth")
@RequiredArgsConstructor
public class AuthController {

    private final AuthService authService;
    private final AuditService auditService;
    private final JwtUtil jwtUtil;

    @PostMapping("/register")
    public ApiResponse<Void> register(@RequestBody @Valid RegisterRequestDTO dto) {
        authService.register(dto);
        return ApiResponse.success("User registered successfully", null);
    }

    @PostMapping("/login")
    public ApiResponse<LoginResponseDTO> login(@RequestBody @Valid LoginRequestDTO dto) {
        return ApiResponse.success("Login successful", authService.login(dto));
    }

    @GetMapping("/validate")
    public ApiResponse<Map<String, Object>> validate(
        @RequestHeader("Authorization") String header) {
        String token = header.startsWith("Bearer ") ? header.substring(7) : header;
        if (!jwtUtil.isValid(token)) {
            return ApiResponse.error("Invalid token");
        }
        Map<String, Object> data = Map.of(
            "valid", true,
            "username", jwtUtil.extractUsername(token),
            "role", jwtUtil.extractRole(token)
        );
        return ApiResponse.success("Token is valid", data);
    }

    // ... user CRUD, audit endpoints
}

```

The `/iam/*` endpoints (profile, change password, preferences, login history) are typically a separate `IamController.java` that handles user-self-service operations and reads `SecurityContextHolder` to get the current user.

11. application.properties

properties

```
server.port=9099
spring.application.name=auth-service

# Eureka
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

# DB
spring.datasource.url=jdbc:mysql://localhost:3306/acquirerx_auth?createDatabaseIfNot
spring.datasource.username=root
spring.datasource.password=kail3114
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# JWT (must match gateway exactly)
jwt.secret=YourSuperSecretKeyForJWTTokenGenerationMustBeAtLeast256BitsLong2024Acquir
jwt.expiration=864000000

# Servlet path prefix
server.servlet.context-path=/api/v1
```

Key config notes:

- `createDatabaseIfNotExist=true` — auto-creates the schema on first run
- `ddl-auto=update` — creates/alters tables based on entity changes (dev convenience). In production, use `validate` + Flyway/Liquibase migrations
- `server.servlet.context-path=/api/v1` — every endpoint is automatically prefixed with `/api/v1`, so controllers don't need to repeat it

12. The Full Login Flow (End-to-End)

1. Browser → POST `http://localhost:9090/api/v1/auth/login`
`{ "username": "admin", "password": "admin123" }`
2. Gateway:

- Path `/api/v1/auth/login` is in open-paths → skip JWT
- Rate limit tier = strict (5/min) → check bucket → OK
- Forward to AUTH-SERVICE via Eureka

3. Auth-service receives the request:

- `SecurityConfig` allows `/api/v1/auth/login` (`permitAll`)
- `JwtFilter` sees no token, skips authentication
- `AuthController.login(dto)` is invoked
- `AuthService.login(dto)`:
 - a. Find user by username
 - b. Check status=ACTIVE
 - c. `BCrypt.matches(provided, hashed)` → if mismatch, throw `BadCredentials`
 - d. `JwtUtil.generateToken(username, role)` → returns signed JWT
 - e. Update `lastLoginAt`
 - f. `AuditService.logAction(username, "LOGIN_SUCCESS", ...)`
- Returns `LoginResponseDTO(token, username, role, message)`

4. Response flows back through gateway → frontend

5. Frontend:

- axios receives response
- Saves token: `localStorage.setItem('ax_token', token)`
- `AuthContext` updates state
- Redirects to `/dashboard` (or whatever `returnTo` was)

6. All subsequent requests:

- axios interceptor adds `Authorization: Bearer <token>`
- Gateway validates token via `JwtAuthenticationFilter`
- Gateway adds `X-Username`, `X-User-Role` headers
- Forwards to downstream service

13. Why This Design Is Robust

- **Stateless** — auth-service can be scaled horizontally; no session stickiness needed
- **No password leakage** — `BCrypt` hashes are one-way; even DB dump doesn't reveal passwords
- **Defense in depth** — gateway validates JWT, but auth-service's filter would too if hit directly
- **Audit trail** — every sensitive action logged

- **Role-based** — single `role` claim is propagated everywhere via JWT, no extra DB lookups in downstream services

14. Common Pitfalls

Pitfall	Symptom	Fix
<code>jwt.secret</code> differs between auth and gateway	All requests get 401 after login	Copy the secret exactly
Forgot to add <code>/api/v1/auth/login</code> to gateway open-paths	Login itself fails because no token yet	Add it to <code>gateway.jwt.open-paths</code>
<code>spring.application.name</code> not set	Service doesn't appear in Eureka	Set the name
Using <code>parseClaimsJws</code> with wrong key	<code>SignatureException</code>	Same key for sign + verify
Forgetting <code>BCryptPasswordEncoder</code> bean	<code>NoSuchBeanDefinitionException</code>	Register the bean in SecurityConfig
Returning JWT in URL query param	Token leaks via referer header	Always return in JSON body

15. Summary

Aspect	Detail
Purpose	Identity, JWT, audit
Port	9099
DB	<code>acquirerx_auth</code> (2 tables: <code>app_user</code> , <code>audit_log</code>)
Tech	Spring Security 6 + BCrypt + JWT + JPA
Public endpoints	<code>/auth/login</code> , <code>/auth/register</code> , <code>/auth/validate</code>

Aspect	Detail
Protected endpoints	All <code>/iam/*</code> , user CRUD, audit
JWT structure	<code>sub=username</code> , <code>role</code> claim, HS256, 24h expiry
Audit	Append-only log of all sensitive actions
Roles	ADMIN, RISK, DISPUTES, RECON, MERCHANT_OPS

Next: Merchant Service — onboarding, KYC, pricing, settlement profiles, stores.